

LSM6DSOX IIO Linux Driver Development

Giacomo Di Clerico, Lorenzo D'Ortona

Academic Year 2025/2026

Contents

1	Project data	1
2	Project description	1
2.1	Importance for the AOS course	1
2.2	Design and implementation	2
2.2.1	Comparison with the Official st_lsm6dsx Kernel Driver	2
3	Project outcomes	3
3.1	Concrete outcomes	3
3.1.1	Device Instantiation and Direct Mode	3
3.1.2	Triggered Buffer and Streaming	3
3.2	Learning outcomes	3
3.3	Existing knowledge	3
3.4	Problems encountered	4
3.5	Code Quality and Kernel Compliance	4
4	Honor Pledge	4

1 Project data

- Project supervisor(s): Prof. Vittorio Zaccaria
- Describe in this table the group that is delivering this project:

<u>Last and first name</u>
Di Clerico Giacomo
D'Ortona Lorenzo

- Describe here how development tasks have been subdivided among members of the group:
 - Both members worked together on all phases of the project, including hardware setup, driver implementation (I2C, Regmap, IIO core), and testing. Tasks were shared equally to ensure both gained full understanding of the kernel environment.
- Links to the project source code: https://github.com/MishowHD/lsm6dsox_driver

2 Project description

This project documents the development of a Linux IIO driver for the ST LSM6DSOX IMU, as part of the **Advanced Operating Systems (AOS)** course. We selected the LSM6DSOX from the **Adafruit** catalog due to its advanced feature set, which includes an integrated temperature sensor, Machine Learning Core (MLC), and Finite State Machines (FSM). While these advanced features were the primary motivation for our choice, they were considered out of scope for this initial implementation, which focuses on the core IMU functionality.

2.1 Importance for the AOS course

This project is highly relevant to the AOS course as it provides practical experience with Linux kernel internals and the development of kernel modules. It directly applies concepts such as:

- **Kernel-User Space Interfacing:** Using `sysfs` and `devfs` for data exchange.
- **Synchronization:** Managing concurrency in kernel space (transitioning from mutexes to the Regmap locking model).
- **Subsystem Architecture:** Understanding the layering of the Industrial I/O (IIO) framework.
- **Interrupt Handling:** Implementing hardware-driven data acquisition via triggered buffers.

The project helps bridge the gap between theoretical operating system concepts and the real-world complexities of hardware-software interaction in a modern monolithic kernel.

2.2 Design and implementation

The development was carried out on a **Debian 13** virtual machine managed via **Virtual Machine Manager** (QEMU/KVM). To streamline the workflow, we used a shared directory between the host and the guest using **virtiofs**. This allowed us to compile the module on the guest while keeping the source code and the git repository on the host.

The hardware setup used:

- **Sensor:** LSM6DSOX on an Adafruit breakout board.
- **Adapter:** **MCP2221A** (USB-to-I2C/GPIO bridge). This allowed us to interface the I2C sensor directly with a standard PC via USB, bypassing the need for an embedded development board (like a Raspberry Pi) and enabling development within a virtualized environment.
- **Cabling:** A **STEMMA QT** (JST SH 4-pin) cable connected the sensor to the adapter.

The driver followed an incremental implementation:

1. **Basic I2C:** Setup of the probe function and `WHO_AM_I` check.
2. **IIO Core:** Implementation of `read_raw` for scaled accelerometer and gyroscope data.
3. **Regmap:** Refactoring to use the `regmap` API for kernel compliance and simplified I/O management.
4. **Triggered Buffers:** Configured the `INT1` pin for hardware-driven data streaming (emulated via software `hrtimer` in the virtual environment due to the lack of hardware IRQ routing through the USB adapter).

2.2.1 Comparison with the Official `st_lsm6dsx` Kernel Driver

The mainline Linux kernel includes an official driver for the LSM6DSX family, located at `drivers/iio/imu/st_lsm6dsx/`. A comparison between that implementation and ours highlights the trade-offs between a production-grade, general-purpose driver and a focused, educational one.

Architecture and Scope The official driver is a **multi-file subsystem** split across several source files (`st_lsm6dsx_core.c`, `st_lsm6dsx_i2c.c`, etc.), supporting over a dozen variants. Our driver is a **single-file, single-device** implementation, prioritizing clarity and traceability at the cost of portability.

Data Acquisition The official driver leverages the sensor's on-chip **hardware FIFO** (up to 3 kB) and drains it via watermark interrupts. Our driver implements a standard **IIO triggered buffer**: on each trigger event, the handler performs a live `regmap_bulk_read` and pushes data to the `kfifo`. While our approach is functionally correct and easier to implement, the official one is significantly more power-efficient.

Configurability In our driver, the ODR and full-scale range are **hardcoded** at probe time (104 Hz and 2 g / 250 dps). The official driver exposes these as **IIO attributes** (`in_accel_sampling_frequency`, `in_accel_scale`), allowing users to reconfigure the sensor at runtime without reloading the module.

Summary Table

Feature	Our Driver	Official <code>st_lsm6dsx</code>
Transport	I2C only	I2C + SPI
Multi-device support	LSM6DSOX only	15+ variants
ODR / FS configuration	Hardcoded at probe	Runtime via sysfs
Data acquisition	Triggered buffer (kfifo)	Hardware FIFO + watermark IRQ
Device enumeration	Manual (<code>new_device</code>)	Device Tree / ACPI
Advanced Features	x	Step counter, MLC, FSM

3 Project outcomes

3.1 Concrete outcomes

The primary artifact is a functional Linux kernel module (`lsm6dsx_driver.ko`) that exposes the sensor's capabilities through the Industrial I/O (IIO) framework.

3.1.1 Device Instantiation and Direct Mode

Since the sensor is not defined in a Device Tree in our virtualized environment, we manually instantiate it on the I2C bus:

```
1 echo lsm6dsx 0x6a | sudo tee /sys/bus/i2c/devices/i2c-1/new_device
```

Once registered, the driver exposes raw data attributes through sysfs. Verification is performed by reading these attributes, which triggers the driver's `read_raw` function:

```
1 # Read raw accelerometer X-axis data
2 cat /sys/bus/iio/devices/iio:device0/in_accel_x_raw
3 # Read the scale factor
4 cat /sys/bus/iio/devices/iio:device0/in_accel_scale
```

3.1.2 Triggered Buffer and Streaming

To validate high-speed data acquisition without hardware interrupts, we used a software-based trigger (`iio-trig-hrtimer`):

1. Trigger Creation:

```
1 sudo mkdir /sys/kernel/config/iio/triggers/hrtimer/finto
2
```

2. **Buffer Configuration:** We enabled the desired channels and bound the trigger to the device.

3. **Data Validation:** Enabling the buffer allowed us to stream data through the `kfifo`, verified via `hexdump`:

```
1 echo 1 | sudo tee /sys/bus/iio/devices/iio:device0/buffer/enable
2 sudo cat /dev/iio:device0 | hexdump -C
3
```

The output confirmed a 12-byte payload per sample (including active axes and an 8-byte timestamp), demonstrating successful hardware-software synchronization.

3.2 Learning outcomes

The development process provided deep insights into the Linux kernel's IIO subsystem, the `regmap` API, and kernel-space synchronization. We also learned how to interact with kernel tooling like `i2c-tools` for validation. In terms of unanswered questions, while we successfully implemented a triggered buffer, the full integration of the sensor's internal hardware FIFO remains an area for future exploration. The knowledge acquired regarding concurrency and hardware-software interfacing will be highly valuable in our future careers, especially for system-level programming and embedded systems development.

3.3 Existing knowledge

The **Operating Systems** course provided the essential theoretical foundation. Our practical background was further expanded during the **laboratory sessions**, where we developed trivial drivers, gaining initial familiarity with the kernel environment and the module lifecycle. Since we had never written a complex kernel driver, we supplemented this by extensively studying the Linux kernel documentation. This project was a practical application of OS theory to a low-level task. Regarding suggestions for the AOS course, we particularly enjoyed the practical laboratory sessions. We believe that incorporating even more hands-on labs, perhaps focusing on kernel module debugging techniques, would further enhance the learning experience.

3.4 Problems encountered

- **IIO Trigger Availability:** Hardware IRQs were unavailable in our initial VM setup. We faced difficulties setting up hardware-based triggers and initially fell back to polling mode. To test the triggered buffer functionality, we switched our environment to **Debian 13**, which provided better support for the `iio-trig-hrtimer` module, allowing us to instantiate a virtual high-resolution timer as a trigger.
- **Virtiofs Permissions:** Configuring the shared directory with the correct permissions for kernel module compilation was a preliminary step required to manage the source code between the host and the guest smoothly.
- **Debugging:** Without a logic analyzer, we relied heavily on `dmesg` to verify if the `probe` function was successful and to identify that hardware interrupts were not being routed correctly in the VM.

3.5 Code Quality and Kernel Compliance

To ensure that the driver meets the rigorous standards required for inclusion in the Linux kernel, we performed static analysis and style verification. The primary tool used was `checkpatch.pl`, the official script used by kernel developers. We iteratively refined the code to adhere to the **Linux Kernel Coding Style**, focusing on:

- **Formatting:** Proper use of tabs for indentation and adhering to the 100-column line length limit.
- **Naming Conventions:** Strict adherence to `snake_case` for functions, variables, and structures.
- **Kernel Utilities:** Extensive use of kernel-specific macros and managed resources (e.g., `devm_*` functions for automatic resource cleanup, `ARRAY_SIZE()` for array bounds, and `PTR_ERR()` for error handling) to ensure robust and idiomatic code.

4 Honor Pledge

I/We pledge that this work was fully and wholly completed within the criteria established for academic integrity by Politecnico di Milano (Code of Ethics and Conduct) and represents my/our original production, unless otherwise cited.

I/We also understand that this project, if successfully graded, will fulfill part B requirement of the Advanced Operating System course and that it will be considered valid up until the AOS exam of Sept. 2026.

Giacomo Di Clerico
Lorenzo D'Ortona